

# Guidelines

8 June 2024

1. ***Keep your cell phones in silent mode and hand it over to an instructor.***
2. Do not scribble anything on this sheet or problem statement. If you need blank sheets for temporary work, please check with the instructor.
3. Do not use ChatGPT (or its variants in any form) to complete your exercises.
4. You may use your favourite IDE or editor to write programs.
5. Ensure that code you write has appropriate indentation, variable and function names and compiles without any error.
6. Test your program with a few test cases and test your program. Your selection of test cases should cover regular happy paths, invalid inputs and edge cases. You will demonstrate the working of your program with test cases during the follow-up discussion.
7. While dealing with file operations, neither assume a specific number of lines nor number of characters per line. Your program must handle exceptions in case there are hardware limitations in terms of memory.
8. While dealing with input data or command line arguments, your program must gracefully handle the exceptions and throw an error message if it cannot proceed further.
9. You may use online documentation and tutorials for references but do not blindly copy the code.
10. Make *reasonable* assumptions wherever you do not have specific details you need to complete your exercise. You may put comments at the beginning of the code listing your assumptions, if any. Reasonableness of your assumptions will be discussed during the follow-up discussion.
11. Look out for specific instructions within the problem statements written in *italics*.
12. ***Return all sheets, including your temporary work, to the instructor.***

**Write a program mysqroot to find a square root of a number using Newton's method.**

Newton's method to derive a square root of a positive number is an iterative method by applying the following formula multiple times.

$$z \text{ -= } (z*z - x) / (2*z)$$

WHERE

"x" is an input number for which square root have to be calculated

"z" is the square root of a number. You may start as z=1 as initial guess and iterate it till you arrive at the square root of of a number with adequate accuracy

Use max 25 iterations.

If between any two iterations, the difference is less than or equal to 0.001, then the program should stop at that stage instead of completing max iterations mentioned above.

**Part A:** This program will take the 1st argument on the command line, and generates output on STDOUT. Use 4 decimal points for the output.

mysqroot <input>

Expected output:

<input> <result>

Where the result could be "incorrect number", "number not provided" or actual output.

e.g.

For "mysqroot 2", output will be 2 1.4142

For "mysqroot 5.00", output will be 5.00 2.2360

For "mysqroot -1.9", output will be -1.9 Incorrect number

... and so on

*Proceed to Part B only after PART A is completed.*

**Part B:** Make a copy of the above program as mysqroot2 and change it to accept numbers from a file, one per line instead of a number. Output will be as many lines as input, as per the same format mentioned above.

Write a program mycat to accept file names as arguments and generate output on STDOUT equal to "cat file1 file2 file3 ..." command on Linux, where file1, file2 etc are actual filenames.

Write a program myuniq to accept file name as 1st argument and generate output on STDOUT equal to "uniq -c file1" command on Linux, where file1, file2 etc are actual filenames.

Write a program `mysort` to accept file name as 1st argument and generate output on STDOUT equal to "`sort file1`" command on Linux, where `file1`, `file2` etc are actual filenames.

Write a program `mygrep` to accept "text" to be searched as 1st argument, file name as 2nd argument and generate output on STDOUT equal to "`grep -i 'text' file1`" command on Linux, , where file1, file2 etc are actual filenames.

Write a program `myfixed2csv` which will convert a fixed width file to a CSV file. Filename, number of columns and length of each column will be provided as an argument. For each field, once parsed, remove leading and trailing spaces and leading zeros, and enclose it within double quotation marks. Maintain the sequence of fields extracted.

Usage:

```
myfixed2csv <fixed-width-file> <n-columns> <length1> <length2> ...
```

Here is an example of a possible fixed width file which has to be converted to a CSV file.

```
user1      00102357      Magesh T
user21     02200370      Nina G
```

```
myfixed2csv file1 4 10 4 4 15
```

WHERE

- `fixed-width-file` is name of file, in above case `file1`
- `n-columns` - number of columns, in above case, 4
- `length1` - length1 of 1st field, in above case 10, with trailing spaces
- `length2` - length of 2nd field, in above case 4, with leading zeros
- `length3` - length of 2nd field, in above case 4, with leading zeros
- `length4` - length of 2nd field, in above case 15, with leading spaces

should generate output as

```
"user1", "10", "2357", "Mangesh T"
"user21", "220", "370", "Nina G"
```

### Q. get\_schedule - PART 1

Write a program `get_schedule` to read a file, one line per file describes time slot and few additional attributes. This file will have max 100 rows hence use an array instead of dynamically allocated memory.

File starts with a header line followed by one or more lines to describe details. Any line starting with '#' character shall be ignored during processing. Start Time and End time will be in HH24:MM format e.g. 00:30 is 30 min past midnight while 14:45 is 2:30pm. You need not worry about the timezone in this code.

Example file is provided below. An actual file may have less or more records, and can have different time ranges.

```
#start-time,end-time,loop-interval,max-limit,max-delay
00:00,09:00,3600,10000,600
09:01,13:00,1800,1000,600
13:01,14:30,900,1000,600
14:31,15:00,5,1000,60
15:01,23:30,3600,10000,1800
#23:31,23:59,3600,10000,1800
```

internal data structure to describe each line in above file format

```
from: int, // HH*60 + MM for efficient comparison
to: int, // HH*60 + MM for efficient comparison
interval: int,
maxlimit: int,
maxdelay: int
line: int // line number at which this entry was found
entry: string // actual string which was parsed to fill-up this data structure
```

```
get_schedule <time slot-file> <time-of-day>
```

WHERE

```
time slot-file: file in the format described above
time-of-day: time in HH24:MM format e.g. 15:27
```

Program will read time slot-file and populate an array of data structure described above, and then produce the matching line within which <time-of-day> falls

```
e.g. for above file format, if time-of-day is 15:27, it will generate on STDOUT
15:27 15:01,23:59,3600,10000,1800
e.g. for above file format, if time-of-day is 10:43, it will generate on STDOUT
10:43 09:01,13:00,1800,1000,600
```

It is possible a matching time slot may not be available e.g. 23:46. In that case, the program will exit with status 1, and print on STDERR.

```
23:46 No matching time slot found
```



### Q. get\_schedule - PART 2

Modify `gen_schedule` to perform validations and handle out of order time slots e.g.

Validations:

(Replace X1, XX, Y1, Y2 etc with actual line number)

1. It will print an error on STDERR for every line which does not match HH24:MM format, or other values which must be positive integers. Error format will be  
Line XX: from time is not correct, max-limit is not valid
2. It will check for overlapping time slots, if any time slot is found overlapping, it will generate an error on STDERR as  
Line XX and Line YY time slots are overlapping  
Line X1 and Line Y2 time slots are overlapping

Once it accepts a file having valid content, if lines are out of order, it will sort it in an internal array, so that subsequent searches are faster. Use `qsort(3)` from the C library. Do not use homegrown sort implementation.

### Q. get\_schedule - PART 3

Modify above code to accept one or more <time-of-day> on command line and print a matching slot information for each such time-of-day as per the format described earlier.

```
get_schedule <time slot-file> <time-of-day> [ <time-of-day> ... ]
```

WHERE

time slot-file: file in the format described above

time-of-day: one or more of times in HH24:MM format e.g. 15:27 23:35 19:20